CSS NOTES
For Beginners

Theory • Selectors • Properties • Code Examples • Practice Tasks

Topics covered

CSS introduction and implementation, element/class/id selectors, box model, border, margin, padding, colors, font and text properties, Google Fonts, Flexbox, display, and position.

Beginner-friendly learning notes

Table of Contents

- 1. Introduction to CSS
- 2. Ways to Implement CSS
- 3. CSS Selectors: Element, Class, and ID
- 4. Colors and Backgrounds
- 5. Font and Text Properties
- 6. Google Fonts
- 7. CSS Box Model
- 8. Border, Margin, and Padding
- 9. Display Property
- 10. Flexbox
- 11. Position Property
- 12. Practice Questions and Lab Tasks

How to use these notes: Create an index.html file and a style.css file. Type each example, open the HTML file in a browser, and experiment with the CSS values.

1. Introduction to CSS

What is CSS?

CSS stands for Cascading Style Sheets. It is used to control the presentation and layout of HTML documents, including colors, fonts, spacing, borders, and responsive layouts.

HTML defines the structure and content of a page; CSS controls how that content looks and is arranged.

Basic CSS syntax

```
selector {
    property: value;
}
```

A selector chooses the HTML element(s) to style. A property identifies what to change, and a value specifies the desired setting.

```
p {
    color: blue;
    font-size: 18px;
}
```

- p is the selector.
- color and font-size are properties.
- blue and 18px are values.
- Each declaration normally ends with a semicolon.

2. Ways to Implement CSS

CSS can be added to HTML in three main ways: inline, internal, and external CSS.

2.1 Inline CSS

Inline CSS is written in the style attribute of an HTML element. It is useful for quick demonstrations, but can become difficult to maintain across a large website.

```
<h1 style="color: blue; font-size: 30px;">
    Welcome to CSS
</h1>
```

2.2 Internal CSS

Internal CSS is written inside a style element, usually in the head section of the HTML document. It is useful when styles apply to one page.

```
<!DOCTYPE html>
<html>
<head>
    <title>Internal CSS</title>
    <style>
        h1 {
            color: navy;
        }

        p {
            font-size: 18px;
        }
    </style>
</head>
<body>
    <h1>Welcome</h1>
    <p>This page uses internal CSS.</p>
</body>
</html>
```

2.3 External CSS

External CSS is written in a separate .css file and connected to HTML using the link element. This is usually the recommended approach for multi-page websites.

```
<!-- index.html -->
<!DOCTYPE html>
```

```
<html>
<head>
    <title>External CSS</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Welcome</h1>
    <p>This page uses external CSS.</p>
</body>
</html>

/* style.css */
h1 {
    color: navy;
}

p {
    font-size: 18px;
}
```

Method	Where CSS is written	Typical use
Inline	Inside an element's style attribute	Small one-off demonstration
Internal	Inside a style element in the HTML file	Styles for one page
External	In a separate .css file	Reusable styles across pages

Good practice: Prefer external CSS for maintainability. When multiple rules apply, the browser resolves them using the cascade, specificity, source order, and other CSS rules.

3. CSS Selectors: Element, Class, and ID

A selector identifies which HTML elements should receive a CSS rule.

3.1 Element selector

An element selector selects all elements with the given HTML tag name.

```
/* Styles every paragraph */
p {
    color: #334155;
    font-size: 16px;
}
```

3.2 Class selector

A class selector begins with a dot (.) and selects elements whose class attribute contains that class. The same class can be used on many elements.

```
/* HTML */
<p class="highlight">Important paragraph</p>
<h2 class="highlight">Important heading</h2>

/* CSS */
.highlight {
    color: darkred;
    background-color: #fff1f2;
}
```

3.3 ID selector

An ID selector begins with # and selects the element with the matching id. An id should be unique within the HTML document.

```
<!-- HTML -->
<h1 id="main-title">My Website</h1>

/* CSS */
#main-title {
    color: navy;
    text-align: center;
}
```

Selector type	CSS syntax	Example
Element	tag name	p { color: blue; }
Class	.className	.card { padding: 16px; }
ID	#idName	#header { background-color: navy; }

Combined example

```
<!DOCTYPE html>
<html>
<head>
    <style>
        p { color: #334155; }
        .highlight { background-color: #fef3c7; }
        #page-title { color: #1d4ed8; }
    </style>
</head>
<body>
    <h1 id="page-title">CSS Selectors</h1>
    <p>This paragraph uses the element selector.</p>
    <p class="highlight">This paragraph also uses a class selector.</p>
</body>
</html>
```

Quick memory tip: element selector = tag name; class selector = dot (.) plus class name; ID selector = hash (#) plus id value.

4. Colors and Backgrounds

CSS colors can be written using named colors, hexadecimal values, RGB, RGBA, HSL, or HSLA.

Format	Example	Explanation
Named color	color: red;	Uses a predefined CSS color name.
Hexadecimal	color: #2563eb;	Uses a hex color value.
RGB	color: rgb(37, 99, 235);	Defines red, green, and blue channels.
RGBA	background: rgba(0,0,0,0.5);	Adds an alpha channel for transparency.
HSL	color: hsl(221, 83%, 53%);	Uses hue, saturation, and lightness.

Common color and background properties

Property	Purpose
color	Sets text color.
background-color	Sets the background color.
background-image	Sets a background image.
background-repeat	Controls whether a background image repeats.
background-position	Sets the background image position.
background-size	Controls background image sizing, e.g. cover.
opacity	Controls the opacity of an entire element and its contents.

```
body {
    background-color: #f1f5f9;
    color: #1e293b;
}

.card {
    background-color: white;
    border: 1px solid #cbd5e1;
}
```

```
.hero {
    background-image: url("images/banner.jpg");
    background-repeat: no-repeat;
    background-position: center;
    background-size: cover;
}
```

Use sufficient contrast between text and background so that content is readable.

5. Font and Text Properties

Font properties

Property	Purpose	Example
font-family	Specifies the typeface or fallback list.	font-family: Arial, sans-serif;
font-size	Sets the size of text.	font-size: 18px;
font-weight	Controls thickness, commonly 400 or 700.	font-weight: 700;
font-style	Uses normal or italic text.	font-style: italic;
line-height	Controls the height of a line box.	line-height: 1.6;
font-variant	Controls variants such as small caps.	font-variant: small-caps;

Text properties

Property	Purpose	Example
text-align	Aligns text horizontally.	text-align: center;
text-decoration	Adds or removes underline/overline/line-through.	text-decoration: underline;
text-transform	Changes capitalization visually.	text-transform: uppercase;
text-indent	Indents the first line of text.	text-indent: 24px;
letter-spacing	Adjusts spacing between characters.	letter-spacing: 1px;
word-spacing	Adjusts spacing between words.	word-spacing: 4px;
text-shadow	Adds a shadow to text.	text-shadow: 1px 1px 2px #aaa;
white-space	Controls whitespace and wrapping.	white-space: nowrap;

```
h1 {
    font-family: Arial, sans-serif;
    font-size: 32px;
    font-weight: 700;
    text-align: center;
    text-transform: uppercase;
}

p {
    font-size: 16px;
    line-height: 1.6;
    letter-spacing: 0.2px;
    color: #334155;
}

a {
    text-decoration: none;
}
```

Common units include px (pixels), rem (relative to the root font size), em (relative to the relevant font size), and % (percentage). For scalable layouts, rem and relative units are often useful.

6. Google Fonts

Google Fonts is a library of web fonts that can be used to customize typography. A font can be loaded using a link in the HTML head or an @import rule in CSS. The HTML link approach is commonly preferred.

Step 1: Add the font link in HTML

```
<!DOCTYPE html>
<html>
<head>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
          rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>This paragraph uses Poppins.</p>
</body>
</html>
```

Step 2: Use the font in CSS

```
body {
    font-family: "Poppins", sans-serif;
}

h1 {
    font-weight: 700;
}

p {
    font-weight: 400;
}
```

- Choose the font and weights you need from the Google Fonts website.
- Use a generic fallback such as sans-serif in case the web font does not load.
- An internet connection is usually needed to fetch the hosted font unless it is self-hosted.
- Only request the weights and styles needed to avoid unnecessary downloads.

7. CSS Box Model

Every element is represented as a rectangular box. The CSS box model describes how its content, padding, border, and margin fit together.

Part	Meaning
Content	The text, image, or other content inside the element.
Padding	Space between the content and the border.
Border	The line surrounding the padding and content.
Margin	Space outside the border, separating the element from others.

```
.box {
    width: 200px;
    padding: 20px;
    border: 5px solid navy;
    margin: 15px;
}
```

With the default content-box model, the width of the content is 200px. The total border-box width is 200 + 20×2 padding + 5×2 border = 250px, not including margins.

box-sizing

```
/* Recommended common reset */
* {
    box-sizing: border-box;
}

.box {
    width: 200px;
    padding: 20px;
    border: 5px solid navy;
}
```

With box-sizing: border-box, the declared width includes the content, padding, and border. Margins remain outside the declared width.

> Remember the order from inside to outside: Content → Padding → Border → Margin.

8. Border, Margin, and Padding

8.1 Border

The border property draws a border around an element. It is commonly written as width, style, and color.

```
.card {
    border: 2px solid #2563eb;
    border-radius: 10px;
}
```

Property	Purpose
border-width	Sets border thickness.
border-style	Sets the style, e.g. solid, dashed, dotted, double, none.
border-color	Sets the border color.
border-radius	Rounds the corners.
border-top / right / bottom / left	Sets the border on an individual side.

8.2 Padding

Padding is the space between the content and its border. It is inside the border.

```
/* Same padding on all four sides */
.card {
    padding: 20px;
}

/* top, right, bottom, left */
.card {
    padding: 10px 15px 20px 25px;
}

/* vertical then horizontal */
.card {
    padding: 12px 20px;
}
```

8.3 Margin

Margin is the space outside the border. It separates an element from surrounding elements.

```
/* Same margin on all sides */
.card {
    margin: 20px;
}

/* vertical then horizontal */
.card {
    margin: 10px 20px;
}

/* Center a block with a constrained width */
```

```
.container {
    width: 80%;
    max-width: 1000px;
    margin: 0 auto;
}
```

Vertical margins between normal block elements can collapse in certain circumstances. Padding does not collapse in the same way.

Quick comparison

Property	Where it appears	Typical purpose
Border	Around content and padding	Outline or separate a component.
Padding	Inside border	Create breathing room inside a box.
Margin	Outside border	Create space between boxes.

9. Display Property

The display property controls how an element participates in layout.

Value	Behavior
block	Starts on a new line and generally takes available width.
inline	Flows within text; width and height generally do not apply as for block boxes.
inline-block	Flows inline while allowing width and height to be set.
none	Removes the element from layout and rendering.
flex	Creates a flex container for one-dimensional layout.
grid	Creates a grid container for two-dimensional layout.
inline-flex	Creates a flex container that itself flows inline.

```
/* Block */
.block-item {
    display: block;
}

/* Inline */
.inline-item {
    display: inline;
}

/* Inline-block */
.badge {
    display: inline-block;
    padding: 6px 10px;
}

/* Hide from layout */
.hidden {
    display: none;
}
```

display: none removes an element from layout. It is different from visibility: hidden, which hides the element but preserves its layout space.

10. Flexbox

Flexbox (Flexible Box Layout) is a CSS layout system for arranging items in one dimension: a row or a column. It is especially useful for navigation bars, aligned cards, and responsive component layouts.

Flex container and flex items

An element becomes a flex container when it uses display: flex. Its direct children become flex items.

```
<div class="container">
```

```
    <div class="item">Home</div>
    <div class="item">About</div>
    <div class="item">Contact</div>
</div>

.container {
    display: flex;
    gap: 16px;
}

.item {
    padding: 16px;
    background-color: #dbeafe;
}
```

Main Flexbox properties

Property	Purpose / common values
display: flex	Creates a flex container.
flex-direction	row, row-reverse, column, column-reverse.
justify-content	Aligns items along the main axis: flex-start, center, flex-end, space-between, space-around, space-evenly.
align-items	Aligns items along the cross axis: stretch, flex-start, center, flex-end, baseline.
flex-wrap	Controls wrapping: nowrap, wrap, wrap-reverse.
gap	Sets spacing between flex items.
flex-grow	Controls how much an item can grow.
flex-shrink	Controls how much an item can shrink.
flex-basis	Sets an item's initial main size.
align-self	Overrides cross-axis alignment for one item.
order	Changes visual ordering of flex items.

Example: Simple navigation bar

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .navbar {
            display: flex;
            justify-content: space-between;
            align-items: center;
            gap: 20px;
            padding: 16px 24px;
            background-color: #0f172a;
        }

        .navbar a {
            color: white;
            text-decoration: none;
        }

        .links {
            display: flex;
            gap: 20px;
        }
    </style>
</head>
<body>
    <nav class="navbar">
        <strong style="color: white;">MySite</strong>
        <div class="links">
            <a href="#">Home</a>
            <a href="#">About</a>
            <a href="#">Contact</a>
        </div>
    </nav>
</body>
</html>
```

Centering an item with Flexbox

```
.parent {
    display: flex;
    justify-content: center; /* main axis */
    align-items: center;     /* cross axis */
    min-height: 250px;
}
```

Responsive wrapping card layout

```
.cards {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
}

.card {
    flex: 1 1 220px;
    padding: 20px;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
}
```

The main axis depends on flex-direction. With row, justify-content works horizontally and align-items vertically. With column, those directions switch.

11. Position Property

The position property controls how an element is positioned in the document. The top, right, bottom, and left properties can offset positioned elements.

Value	Behavior
static	Default position; top/right/bottom/left do not reposition it.
relative	Remains in normal flow and can be offset from its normal position.
absolute	Removed from normal flow and positioned relative to its containing block, often a positioned ancestor.
fixed	Removed from normal flow and positioned relative to the viewport in typical cases.
sticky	Behaves like normal flow until a scroll threshold is reached, then sticks within its scroll container constraints.

11.1 Static

```
.box {
    position: static;
}
```

Static is the default positioning behavior.

11.2 Relative

```
.box {
    position: relative;
    top: 10px;
    left: 20px;
}
```

The element is offset from its normal position while its original layout space remains.

11.3 Absolute

```
.parent {
    position: relative;
    height: 180px;
    border: 1px solid #94a3b8;
}

.child {
    position: absolute;
    top: 10px;
    right: 10px;
}
```

The absolutely positioned child is positioned relative to its containing block, commonly the nearest ancestor that establishes its positioning context.

11.4 Fixed

```
.help-button {
    position: fixed;
    right: 20px;
    bottom: 20px;
}
```

A fixed element is commonly used for a floating help button or a back-to-top control. Certain ancestor properties can affect its containing block.

11.5 Sticky

```
.section-title {
    position: sticky;
    top: 0;
    background-color: white;
}
```

A sticky element needs a scroll context and an offset such as top: 0 to know when to stick. Its behavior is constrained by its scrolling container.

z-index

The z-index property controls stacking order for elements in applicable stacking contexts. A higher value does not always place an element above everything because stacking contexts affect how elements are layered.

```
.modal {
    position: fixed;
    z-index: 1000;
}
```

Use relative on a parent when you want an absolute child to be positioned within that parent. Remember that absolute and fixed elements are generally removed from normal document flow.

12. Practice Questions and Lab Tasks

A. Theory questions

1. What is CSS? Write its full form.
2. Explain the syntax of a CSS rule.
3. Differentiate between inline, internal, and external CSS.
4. What is an element selector?
5. What is the difference between class and ID selectors?
6. How can CSS colors be represented?
7. Explain any five font or text properties.
8. What are Google Fonts, and how can they be used?
9. Explain the CSS box model.

10. Differentiate between border, margin, and padding.
11. What is the difference between display: none and visibility: hidden?
12. Explain block, inline, and inline-block.
13. What is Flexbox? What is a flex container?
14. Explain justify-content and align-items.
15. What is the difference between flex-direction and flex-wrap?
16. Explain static, relative, absolute, fixed, and sticky positioning.
17. What is z-index?
18. How can a block element be centered using margin?
19. Why is external CSS useful?
20. How does box-sizing: border-box affect width calculations?

B. Practical lab tasks

Lab Task 1: Three ways to apply CSS

- Create a page demonstrating inline, internal, and external CSS.
- Use element, class, and ID selectors.
- Apply color, background-color, and font-size.

Lab Task 2: Question and answer page

- Create five question-answer pairs.
- Make questions red and answers blue.
- Place each pair inside a bordered box using padding and margin.

Lab Task 3: Product cards with Flexbox

- Create three product cards, each with an image, name, price, and button.
- Arrange the cards in a row using display: flex.
- Use gap, padding, border, and border-radius; allow wrapping on smaller screens.

Lab Task 4: Simple navigation bar

- Create a navbar with a logo and Home, About, and Contact links.
- Use Flexbox to align the links.
- Do not add a search box or dropdown.

Lab Task 5: Positioning demonstration

- Create a parent box with position: relative.
- Place a label inside it using position: absolute.
- Create a fixed help button and a sticky heading.

Lab Task 6: Google Fonts and typography

- Load a Google Font such as Poppins.
- Apply different font weights to headings and paragraphs.
- Use text-align, line-height, text-transform, and text-decoration.

Quick Revision

Concept	Key syntax / properties
CSS rule	selector { property: value; }
Inline / internal / external	style attribute / style element / link stylesheet
Element / class / ID selectors	p { } / .card { } / #header { }
Colors	color, background-color, rgb(), hex, hsl()
Font and text	font-family, font-size, font-weight, line-height, text-align
Google Fonts	link stylesheet in head; font-family in CSS
Box model	content → padding → border → margin
Border	border, border-style, border-color, border-radius
Spacing	margin and padding
Display	block, inline, inline-block, none, flex, grid
Flexbox	flex-direction, justify-content, align-items, gap, flex-wrap
Position	static, relative, absolute, fixed, sticky
Stacking	z-index

Final tip: Learn CSS by changing one property at a time and observing the browser. Practice selectors first, then spacing and the box model, followed by Flexbox and positioning.